



Материали за подготовка - База данни

Клас: 11б | Предмет: База данни

Дата на контролната работа: _____

1. Създаване на бази данни и таблици. Първични ключове

1.1. Създаване на база данни

```
CREATE DATABASE име_на_базата  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_general_ci;
```

Важно: CHARACTER SET utf8mb4 позволява използването на български букви и специални символи.

1.2. Използване на база данни

```
USE име_на_базата;
```

1.3. Създаване на таблица

```
CREATE TABLE име_на_таблица (  
    колона1 ТИП_ДАННИ ОГРАНИЧЕНИЯ,  
    колона2 ТИП_ДАННИ ОГРАНИЧЕНИЯ,  
    ...  
);
```

Пример:

```
CREATE TABLE students (  
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

1.4. Първичен ключ (PRIMARY KEY)

- **Определение:** Колона или комбинация от колони, които уникално идентифицират всеки запис в таблицата
- **Свойства:**
 - Не може да има NULL стойности
 - Трябва да е уникална за всеки запис
 - Всяка таблица може да има само един PRIMARY KEY
- **Начини за задаване:**
 - В дефиницията на колоната: `id INT PRIMARY KEY`
 - С AUTO_INCREMENT: `id INT AUTO_INCREMENT PRIMARY KEY`
 - Отделно: `PRIMARY KEY (id)`

1.5. Типове данни (основни)

Тип	Описание	Пример
INT / INT UNSIGNED	Цели числа (положителни с UNSIGNED)	id, брой, количество
VARCHAR(n)	Променлива дължина текст (до n символа)	име, адрес
TEXT	Дълъг текст	описание, бележки
DECIMAL(p,s)	Десетични числа (p=общо цифри, s=след запетая)	цена, тегло
DATE	Дата	рождена дата
TIMESTAMP	Дата и час	created_at, updated_at
BOOLEAN	Булева стойност (TRUE/FALSE)	is_active, is_published

1.6. Ограничения (CONSTRAINTS)

- **NOT NULL:** Колоната не може да бъде празна
- **UNIQUE:** Всички стойности в колоната трябва да са различни
- **DEFAULT:** Задава стойност по подразбиране
- **AUTO_INCREMENT:** Автоматично увеличаване на стойността
- **PRIMARY KEY:** Първичен ключ

2. Извличане на данни от таблици (SELECT)

2.1. Основен синтаксис

```
SELECT колона1, колона2, ...  
FROM име_на_таблица  
[WHERE условие]  
[ORDER BY колона [ASC|DESC]]  
[LIMIT брой];
```

2.2. Извличане на всички колони

```
SELECT * FROM students;
```

2.3. Филтриране с WHERE

Оператори за сравнение:

- `=` - равно
- `!=` или `<>` - различно
- `>`, `<`, `>=`, `<=` - сравнение
- `LIKE` - търсене по шаблон
- `IN` - в списък от стойности
- `IS NULL` / `IS NOT NULL` - проверка за NULL

-- Примери:

```
SELECT * FROM students WHERE id = 1;  
SELECT * FROM students WHERE first_name = 'Иван';  
SELECT * FROM products WHERE price > 100;  
SELECT * FROM students WHERE email IS NOT NULL;  
SELECT * FROM products WHERE price BETWEEN 50 AND 200;
```

2.4. Сортиране с ORDER BY

```
-- Възходящ ред (ASC е по подразбиране)
SELECT * FROM students ORDER BY last_name ASC;

-- Низходящ ред
SELECT * FROM products ORDER BY price DESC;

-- Сортиране по няколко колони
SELECT * FROM students
ORDER BY last_name ASC, first_name ASC;
```

2.5. Ограничаване на резултатите (LIMIT)

```
-- Първите 5 записа
SELECT * FROM students LIMIT 5;

-- Пропускане на първите 10 и показване на следващите 5
SELECT * FROM students LIMIT 10, 5;
```

3. Промяна на структури на таблици (ALTER TABLE)

3.1. Добавяне на колона

```
ALTER TABLE име_на_таблица
ADD COLUMN име_на_колона ТИП_ДАННИ [ОГРАНИЧЕНИЯ];
```

```
ALTER TABLE students  
ADD COLUMN phone VARCHAR(20);
```

3.2. Изтриване на колона

```
ALTER TABLE име_на_таблица  
DROP COLUMN име_на_колона;
```

3.3. Промяна на тип данни на колона

```
ALTER TABLE име_на_таблица  
MODIFY COLUMN име_на_колона НОВ_ТИП_ДАННИ;
```

```
ALTER TABLE students  
MODIFY COLUMN phone VARCHAR(30);
```

3.4. Промяна на името на колона

```
ALTER TABLE име_на_таблица  
CHANGE COLUMN старо_име ново_име ТИП_ДАННИ;
```

3.5. Добавяне на ограничение

```
-- Добавяне на UNIQUE
ALTER TABLE students
ADD UNIQUE (email);

-- Добавяне на PRIMARY KEY
ALTER TABLE students
ADD PRIMARY KEY (id);
```

4. Изтриване на данни и структури

4.1. Изтриване на записи (DELETE)

```
-- Изтриване на всички записи
DELETE FROM име_на_таблица;

-- Изтриване с условие
DELETE FROM students WHERE id = 1;
DELETE FROM products WHERE price < 10;
```

Внимание! DELETE изтрива данни, но не изтрива структурата на таблицата.

4.2. Изтриване на таблица (DROP TABLE)

```
DROP TABLE име_на_таблица;
```

Внимание! DROP TABLE изтрива цялата таблица заедно с данните и структурата!

4.3. Изтриване на база данни (DROP DATABASE)

```
DROP DATABASE име_на_базата;
```

Много внимателно! DROP DATABASE изтрива цялата база данни!

4.4. Изчистване на таблица (TRUNCATE)

```
TRUNCATE TABLE име_на_таблица;
```

TRUNCATE изтрива всички записи, но запазва структурата. По-бързо от DELETE.

5. Релационен модел и проектиране на база данни

5.1. Релационен модел

- **Таблица (Relation):** Колекция от свързани данни, организирани в редове и колони
- **Ред (Tuple/Record):** Един запис в таблицата
- **Колона (Attribute/Field):** Едно поле в таблицата
- **Първичен ключ (Primary Key):** Уникален идентификатор на запис
- **Външен ключ (Foreign Key):** Референция към първичен ключ в друга таблица

5.2. Външен ключ (FOREIGN KEY)

- **Определение:** Колона или комбинация от колони, които реферират PRIMARY KEY в друга таблица
- **Цел:** Поддържане на релационна цялост (referential integrity)
- **Синтаксис:**

```
FOREIGN KEY (колона_в_таблица)
REFERENCES друга_таблица(колона_първичен_ключ)
[ON DELETE действие]
[ON UPDATE действие];
```

Пример:

```
CREATE TABLE orders (
    id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

5.3. Действия при изтриване/промяна (ON DELETE / ON UPDATE)

Действие	Описание
CASCADE	При изтриване/промяна на родителския запис, се изтриват/променят и свързаните записи
SET NULL	При изтриване на родителския запис, външният ключ става NULL
RESTRICT	Забранява изтриването на родителския запис, ако има свързани записи

NO
ACTION

Същото като RESTRICT

```
CREATE TABLE order_items (  
    id INT PRIMARY KEY,  
    order_id INT,  
    product_id INT,  
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE NO ACTION  
    FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE NO ACTION  
);
```

5.4. Типове връзки между таблици

5.4.1. Едно към много (1:N / One-to-Many)

- Един запис в едната таблица може да се свърже с много записи в другата
- Външният ключ се поставя в таблицата "много"
- **Пример:** Един клиент може да има много поръчки

```
-- Таблица customers (едно)  
CREATE TABLE customers (  
    id INT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

```
-- Таблица orders (много)  
CREATE TABLE orders (  
    id INT PRIMARY KEY,  
    customer_id INT,
```

```
FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

5.4.2. Много към много (N:M / Many-to-Many)

- Много записи от едната таблица могат да се свържат с много записи от другата
- Изисква междинна таблица (junction/association table)
- **Пример:** Студенти и курсове (един студент може да учи много курсове, един курс може да има много студенти)

```
-- Таблица students
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100)
);

-- Таблица courses
CREATE TABLE courses (
    id INT PRIMARY KEY,
    title VARCHAR(100)
);

-- Междинна таблица
CREATE TABLE student_courses (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

5.4.3. Едно към едно (1:1 / One-to-One)

- Един запис в едната таблица се свързва с точно един запис в другата

- Външният ключ може да се постави в която и да е от двете таблици
- Често се използва UNIQUE за външния ключ
- **Пример:** Един потребител има точно един профил

```
CREATE TABLE users (  
    id INT PRIMARY KEY,  
    username VARCHAR(50)  
);  
  
CREATE TABLE user_profiles (  
    id INT PRIMARY KEY,  
    user_id INT UNIQUE,  
    bio TEXT,  
    FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

6. Създаване на сложни бази от данни с таблици с връзки

6.1. Стъпки при проектиране

1. **Анализ на изискванията:** Какви данни трябва да съхраняваме?
2. **Идентифициране на обекти:** Кои са основните обекти (таблици)?
3. **Определяне на атрибути:** Какви свойства има всеки обект?
4. **Определяне на връзки:** Как се свързват обектите?
5. **Нормализация:** Премахване на излишни данни
6. **Създаване на таблици:** Първо таблиците без връзки, после с връзки

6.2. Пример: База данни за онлайн магазин

```
-- 1. Таблица за категории (няма връзки)
CREATE TABLE categories (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL
);

-- 2. Таблица за продукти (има връзка към категории)
CREATE TABLE products (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    category_id INT,
    FOREIGN KEY (category_id) REFERENCES categories(id)
);

-- 3. Таблица за клиенти (няма връзки)
CREATE TABLE customers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE
);

-- 4. Таблица за поръчки (има връзка към клиенти)
CREATE TABLE orders (
    id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT NOT NULL,
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

-- 5. Таблица за детайли на поръчките (междинна таблица)
CREATE TABLE order_items (
    id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
```

```
FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE  
FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE  
);
```

6.3. Важни правила

- Винаги създавайте първо таблиците без FOREIGN KEY ограничения
- След това създавайте таблиците, които реферират други таблици
- При изтриване на таблици, първо изтривайте таблиците с FOREIGN KEY
- Използвайте смислени имена за таблици и колони
- Винаги задавайте подходящи типове данни
- Използвайте NOT NULL за задължителни полета

7. Полезни SQL команди за преглед

7.1. Показване на структурата на таблица

```
DESCRIBE име_на_таблица;  
-- или  
DESC име_на_таблица;
```

7.2. Показване на всички таблици в базата

```
SHOW TABLES;
```

7.3. Показване на всички бази данни

```
SHOW DATABASES;
```

8. Често срещани грешки и как да ги избегнем

- **Грешка:** Забравяне на точка и запетая (;)
Решение: Винаги завършвайте SQL командите с ;
- **Грешка:** Използване на неправилни кавички
Решение: Използвайте единични кавички (') за текстови стойности
- **Грешка:** Опит за създаване на FOREIGN KEY към несъществуваща таблица
Решение: Създавайте таблиците в правилния ред
- **Грешка:** Опит за изтриване на таблица, която се използва от друга таблица
Решение: Първо изтрийте таблиците с FOREIGN KEY
- **Грешка:** Забравяне на PRIMARY KEY
Решение: Винаги задавайте PRIMARY KEY за всяка таблица

9. Упражнения за подготовка

Упражнение 1: Създаване на проста база данни

Създайте база данни "library" с таблица "books", която има:

- id (първичен ключ, автоинкремент)
- title (текст, задължително)
- author (текст, задължително)
- isbn (текст, уникален)
- published_year (число)

Упражнение 2: Заявки за извличане

Напишете SQL заявки за:

- Всички книги от определен автор
- Книги, публикувани след 2020 година
- Книги, сортирани по година на публикуване (най-нови първи)

Упражнение 3: Релационна база данни

Разширете базата "library" с:

- Таблица "authors" (id, name, birth_date)
- Променете таблицата "books" да използва author_id (външен ключ) вместо author
- Създайте връзка 1:N между authors и books



Ключови точки за контролната работа

- Знаете как да създавате база данни и таблици
- Разбирате концепцията за PRIMARY KEY и как да го зададете
- Можете да извличате данни с SELECT, WHERE, ORDER BY
- Знаете как да променяте структурата на таблици с ALTER TABLE
- Разбирате разликата между DELETE, DROP и TRUNCATE
- Знаете какво е FOREIGN KEY и как да го използвате
- Разбирате типовете връзки: 1:1, 1:N, N:M
- Можете да създавате сложни бази данни с връзки между таблици

Успех на контролната работа! 🎓